

Assignment - 8

Gaussian Naive Bayes is particularly useful when:

- the dataset contains continuous numerical features
- the features follow a normal (Gaussian) distribution
- A fast and efficient classification model is required

Part A:

I selected the Breast Cancer Wisconsin Dataset from Kaggle and applied the Gaussian Naive Bayes algorithm to classify tumors as malignant or benign.

The model was evaluated using:

- Accuracy → overall correctness of the model
- Precision → how many predicted positives are actually correct.
- Recall → how many actual positives are correctly identified.

Part B:

1. What type of dataset did you select and why?

I selected the Breast Cancer Wisconsin Dataset, which is supervised classification dataset.

- It contains labeled data (benign or malignant)
- All features are numerical, which suits Gaussian Naive Bayes.
- It is clean & widely used for classification tasks.

I chose this dataset because it is ideal for understanding how probabilistic models perform on real-world medical data.

b. what preprocessing steps are necessary before applying gaussian Naive Bayes?

the following preprocessing steps are required:

- Removing irrelevant columns (such as ID)
- converting categorical values into numerical form (eg. M $\rightarrow$ 1, B $\rightarrow$ 0)
- checking for missing values
- splitting the dataset into training & testing sets

These steps ensure that the model receives clean & structured input data.

c. how do you handle missing values in your dataset?
Missing values can be handled using the following methods:

- Removing rows or columns with missing values.
- Replacing with statistical values like mean, median or mode.
- Using interpolation or prediction-based methods

In this dataset, no missing values were found, so no additional handling was required.

d. why is feature scaling usually not required for gaussian Naive Bayes?

Feature scaling is not required because:

- Gaussian Naive Bayes uses mean & variance to calculate probabilities.

- It assumes data follows a normal distribution
- Scaling does not significantly affect probability calculations.

Therefore, the model performs well even without normalization or standardization.

Theory of Naive Bayes:

Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It is called "naive" because it assumes that all features are independent of each other, which is usually not true in real world data but works well in practice.

→ Bayes' theorem likelihood

$$P(C|X) = \frac{P(X|C) P(C)}{P(X)}$$

$\swarrow$ $\searrow$
 Posterior probability prior probability of class
 evidence (const. for all classes)

→ Gaussian Naive Bayes

Gaussian Naive Bayes is used when features are continuous & follow a normal distribution

$$P(x|C) = \frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where: μ (mean) → average value of the feature
 σ^2 (variance) → spread spread of data

II. Implementing theory

1. Import libraries :

- Data handling → pandas, numpy
- Model building → gaussian NB
- evaluation → accuracy, precision, recall.

2. Loading Dataset

pd. read_csv()

↳ This converts dataset into a dataframe for ~~pre~~ processing.

3. Data Preprocessing

steps performed :

- Remove unnecessary column (id)
- convert categorical labels (M/B → 1/0)
- check missing values

4. Features & target separation

x = features

y = target

↳ This separation separates input variables & output labels.

5. Train-test split

train - test - split ()

- training data → used to train model
- testing data → used to evaluate performance
 - ↳ Prevents overfitting

6. Model training

```
model = gaussianNB()
model.fit(X_train, y_train)
```

the model learns:

- mean (μ)
 - variance (σ^2)
- for each feature per of class

7. Prediction

```
y_pred = model.predict(X_test)
```

↳ model predicts class based on probability

8. evaluation Metrics

- accuracy :-
- accuracy - score ()
- overall correct predictions

• Precision :

precision - score ()

• Recall :

Recall - score ()